

Web Service Design and Practice

11. Continuous Deployment

Dr. Kyudong Park

School of Information Convergence



Contents

- CI CD 의 개념
- 서비스 배포를 위한 준비
 - HTTPS 의 적용
 - Domain Name
 - Nginx Proxy Manager
- Jenkins 를 활용한 CD 구축

CI vs. CD

구분	CI (Continuous Integration)	CD (Continuous Delivery)	CD (Continuous Deployment)
주요 목적	코드 변경 사항을 자주 통합하고 자동화된 빌드 및 테스트를 통해 품질 유지	배포할 수 있는 상태까지 자동화하고 수동 승인으로 배포	모든 단계를 자동화하여, 승인 없이 배포까지 자동화
자동화 범위	빌드와 테스트 자동화	빌드, 테스트, 배포 준비까지 자동화	빌드, 테스트, 배포까지 완전 자동화
수동 작업	없음	배포 전 수동 승인 필요	수동 작업 없음, 자동 배포
배포	X	준비 완료 상태까지 자동화, 배포는 수동	배포까지 자동화
피드백 속도	코드 통합 오류에 대한 빠른 피드백 제공	배포 가능한 상태까지 피드백 제공	사용자에게 배포된 상태에 대한 즉각적인 피드백
적용 환경	개발 환경에서 주로 사용	프로덕션 직전까지 자동화	프로덕션 환경까지 자동화

필요한 상황

- CI
 - 코드 변경 사항이 잦고 여러 명의 개발자가 협업할 때
 - 코드를 자주 통합하여 충돌을 줄이고, 초기 단계에서 버그를 빠르게 발견하기 위함
- Continuous Delivery
 - 프로덕션 환경에 배포하기 전에 사람의 승인이 필요한 경우
 - 금융 서비스 등 보수적 환경
- Continuous Deployment
 - 배포 주기가 매우 짧고 빠른 경우
 - **SaaS 애플리케이션이나 웹 서비스에 적합**
 - 변경 사항을 빠르게 사용자가 이용할 수 있음

Contents

- CI CD 의 개념
- **서비스 배포를 위한 준비**
 - HTTPS 의 적용
 - Domain Name
 - Nginx Proxy Manager
- Jenkins 를 활용한 CD 구축

HTTPS의 적용

- HTTP Secure
 - HTTP의 보안 버전으로, 웹 브라우저와 웹 서버 간에 안전하게 데이터를 전송하기 위해 **암호화**를 사용하는 프로토콜
 - HTTP에 SSL/TLS (Transport Layer Security) 를 추가하여 보안을 강화한 통신 프로토콜
 - 해커가 네트워크를 통해 전송되는 민감한 정보를 가로채더라도 데이터를 읽을 수 없게됨
- WebRTC의 배포를 위해서 필수적으로 적용되어있어야함
 - local에서는 WebRTC 동작이 가능함
- HTTPS를 적용하기 위한 요구사항
 - 웹사이트에 SSL/TLS 인증서를 설치해야함
 - 인증서 설치를 위해서는 웹 주소(도메인 네임)가 존재해야함
 - 따라서, IP 주소만으로는 HTTPS를 적용할 수 없음

HTTPS의 적용

- SSL 인증서
 - Secure Sockets Layer 기술을 기반으로 하는 인증서
 - 서버와 클라이언트 간의 암호화된 통신을 가능하게 함
 - 인증서에 포함되는 정보
 - 도메인 이름
 - 인증 기관 정보
 - 인증서 유효 기간
 - 공개 키 (public key)

Domain Name (Hostname)

- IP 는 외우기 어려움 => 직관적인 주소의 필요성
 - www.kw.ac.kr
 - kwidea.com
- 대부분 유료

kwangwoonweb.com	EVENT 16,000원 / 24,000원	선택
kwangwoonweb.co.kr 대한민국	EVENT 15,000원 / 21,000원	선택
kwangwoonweb.kr 대한민국	EVENT 15,000원 / 21,000원	선택
kwangwoonweb.shop	EVENT 500원 / 49,000원	선택
kwangwoonweb.store	EVENT 500원 / 80,000원	선택

Domain Name (Hostname)

- 무료 domain name
 - duckdns
 - 계정 당 5개 domain name 생성 가능
 - 와일드카드 기반 서브도메인을 제공
 - 가끔 연결 속도가 느리거나 끊김 발생
 - noip.com
 - 계정 당 1개 domain name 무료
 - 30일 동안 지속적으로 확인 절차를 거침. 대신 duckdns 보다 안정적임.
 - noip.com 접속해서 확인을 하지 않으면 30일 후 만료됨
 - 와일드카드 기반 서브도메인을 지원하지 않음 (유료에서는 가능)

Update Successful

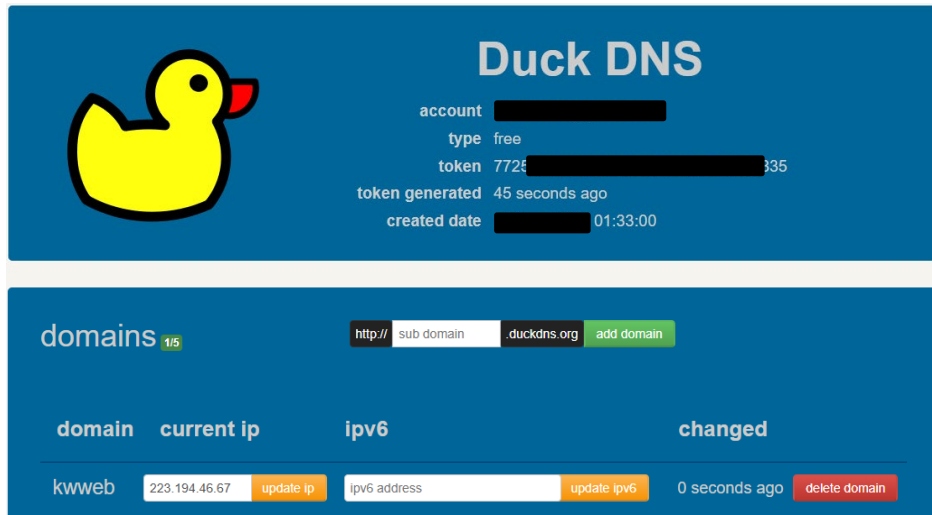
Thank you for confirming your hostname. **rtw-ai.ddns.net** has been updated successfully.

This process helps us keep our Dynamic DNS network free of unused hostnames. Your hostname will expire in 30 days.

Have questions or need help? Feel free to search through our robust [Knowledge Base](#) or, if you need immediate assistance, please open a [support ticket](#).

IP와의 연결 - duckdns

- 나의 IP 주소와 hostname 을 연결 등록
 - 대부분 수 초 내 등록이 완료되며, 즉시 테스트 가능
 - 서버의 IP 주소를 등록해보기
- 절차
 - 회원 가입
 - reCapchar
 - subdomain 입력
 - current ip 수정
- Token 값은 인증서 작업에서 활용됨



IP와의 연결 - noip

- 나의 IP 주소와 hostname 을 연결 등록
 - 대부분 수 초 내 등록이 완료되며, 즉시 테스트 가능
 - 서버의 IP 주소를 등록해보기
- 절차
 - 회원 가입
 - My Account
 - Dynamic DNS
 - No-IP Hostnames
 - Create Hostname
 - 필요한 정보 입력 이후 Submit

+ Create a Hostname

Hostname ⓘ

The name may not be greater than 19 characters.

Domain ⓘ



Record Type ⓘ

☒ DNS Host (A) ⓘ☐ AAAA (IPv6) ⓘ☐ DNS Alias (CNAME) ⓘ☐ Web Redirect ⓘ

[Manage](#) your Round Robin, TXT, SRV and DKIM records.

IPv4 Address ⓘ

Wildcard ⓘ

[Upgrade to Enhanced](#)

to enable wildcard hostnames.

MX Records

[+ Add MX Records](#)

Cancel

Create Hostname with DDNS Key

Create Hostname

Nginx Proxy Manager

- reverse proxy 및 HTTPS 인증서 작업 등을 Web GUI 환경에서 쉽게 수행할 수 있게 돕는 도구
 - another NPM(!)
- 서버에 설치
 - <https://nginxproxymanager.com/setup/>
 - docker 를 활용해서 설치
 - docker 가 설치되어 있지 않다면?
 - <https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository>
 - IP 포트 설정에 유의
 - 서버에서 실제로 활용할 포트와의 연결
 - 방화벽 설정 (포트 오픈)
- 실행
 - \$> sudo docker compose up -d

```
version: '3.8'
services:
  app:
    image: 'jc21/nginx-proxy-manager:latest'
    restart: unless-stopped
    ports:
      # These ports are in format <host-port>:<container-port>
      - '80:80' # Public HTTP Port
      - '443:443' # Public HTTPS Port
      - '81:81' # Admin Web Port
      # Add any other Stream port you want to expose
      # - '21:21' # FTP

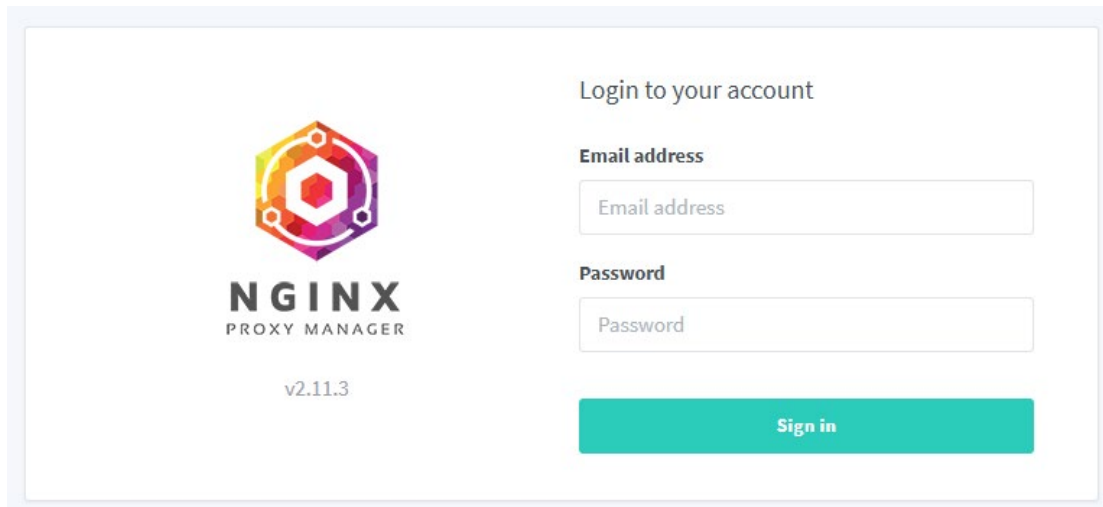
      # Uncomment the next line if you uncomment anything in the section
      # environment:
      #   # Uncomment this if you want to change the location of
      #   # the SQLite DB file within the container
      #   DB_SQLITE_FILE: "/data/database.sqlite"

      # Uncomment this if IPv6 is not enabled on your host
      # DISABLE_IPV6: 'true'

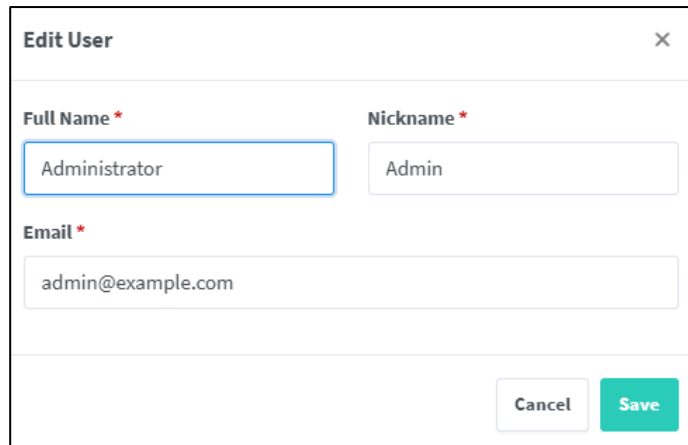
volumes:
  - ./data:/data
  - ./letsencrypt:/etc/letsencrypt
```

Nginx Proxy Manager

- 설치 후 admin port 를 활용하여 웹으로 접속
 - 초기 admin 계정 email : admin@example.com
 - password : changeme



The image shows the login page of Nginx Proxy Manager. On the left is the Nginx Proxy Manager logo, which consists of a colorful hexagonal icon with a white 'N' inside, and the text 'NGINX PROXY MANAGER' and 'v2.11.3' below it. On the right, the text 'Login to your account' is displayed above two input fields: 'Email address' and 'Password'. Below these fields is a large teal button labeled 'Sign in'.



The image shows the 'Edit User' modal window. It has a title bar with 'Edit User' and a close button (X). The modal contains three input fields: 'Full Name *' with the value 'Administrator', 'Nickname *' with the value 'Admin', and 'Email *' with the value 'admin@example.com'. At the bottom right, there are two buttons: 'Cancel' and 'Save'.

Nginx Proxy Manager

- SSL 인증서 작업 수행
 - ① SSL Certificates 메뉴
 - ② Add SSL Certificate 버튼
 - ③ Domain Name 입력
 - subdomain 활용 시에는 와일드카드 활용
 - *.abc.duckdns.org
 - ④ Use a DNS Challenge 설정
 - ⑤ DNS Provider 선택
 - DNS 발급 업체 선택
 - API 토큰값 입력
 - ⑥ 약관 동의

The screenshot shows the 'Add Let's Encrypt Certificate' dialog box. It contains the following fields and options:

- Domain Names ***: A text input field with a warning message below it: "These domains must be already configured to point to this installation".
- Email Address for Let's Encrypt ***: A text input field containing "kdpark@kw.ac.kr".
- Use a DNS Challenge**: A toggle switch that is currently turned on.
- DNS Provider ***: A dropdown menu with the text "Please Choose...". Above this field is a warning message: "This section requires some knowledge about Certbot and its DNS plugins. Please consult the respective plugins documentation.".
- Propagation Seconds**: A text input field. Below it is a note: "Leave empty to use the plugins default value. Number of seconds to wait for DNS propagation.".
- I Agree to the Let's Encrypt Terms of Service ***: A toggle switch that is currently turned off.
- Buttons**: "Cancel" and "Save" buttons at the bottom right.

Nginx Proxy Manager

- Reverse proxy 설정
 - ① Hosts 메뉴
 - ② Proxy hosts 선택
 - ③ Add Proxy Host 버튼 선택
 - ④ Details 탭
 - Domain Names 입력
 - Scheme : http
 - IP : IP 주소 입력 (공유기 내부로 포워딩한다면 내부 IP 도 가능)
 - Forward Port : 해당 주소와 연결할 포트 입력
 - Websockets Support : 소켓 연결 필요 시 설정
 - ⑤ SSL 탭
 - Domain Name 에 맞는 인증서 선택 (미리 발급을 받아야함)
 - Force SSL 설정
 - HTTP/2 Support 설정

The 'Edit Proxy Host' dialog is shown with the 'Details' tab selected. It contains the following fields and options:

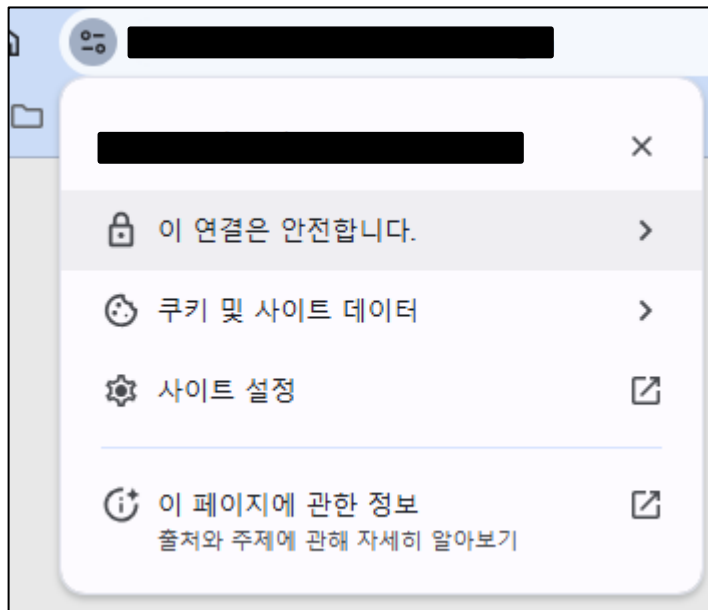
- Domain Names ***: A text input field containing 'artvoice-api.kwidea.com'.
- Scheme ***: A dropdown menu set to 'http'.
- Forward Hostname / IP ***: A text input field containing '223.194.46.134'.
- Forward Port ***: A text input field containing '20225'.
- Cache Assets**: A toggle switch that is currently turned off.
- Block Common Exploits**: A toggle switch that is currently turned off.
- Websockets Support**: A toggle switch that is currently turned off.

The 'Edit Proxy Host' dialog is shown with the 'SSL' tab selected. It contains the following fields and options:

- SSL Certificate**: A text input field containing '*.kwidea.com, kwidea.com'.
- Force SSL**: A toggle switch that is currently turned on.
- HTTP/2 Support**: A toggle switch that is currently turned on.
- HSTS Enabled**: A toggle switch that is currently turned off, with a help icon.
- HSTS Subdomains**: A toggle switch that is currently turned off.
- Buttons**: 'Cancel' and 'Save' buttons at the bottom right.

Nginx Proxy Manager

- 접속 시 https 로 연결되는지 확인



Contents

- CI CD 의 개념
- 서비스 배포를 위한 준비
 - HTTPS 의 적용
 - Domain Name
 - Nginx Proxy Manager
- **Jenkins** 를 활용한 CD 구축

Jenkins

- **오픈 소스 자동화 서버**로, 소프트웨어 개발 프로젝트를 지속적으로 통합하고 쉽게 배포할 수 있게 돕는 도구
- **작동 방식**
 - **코드 변경** : 개발자가 코드 저장소에 코드를 푸시하면 Jenkins가 이를 감지함
 - github 에서는 Webhook 을 통해 Jenkins 로 알림
 - **자동 빌드 및 테스트** : Jenkins는 프로젝트를 자동으로 빌드하고, 사전에 정의된 테스트를 실행함
 - **결과 피드백** : 성공 여부를 개발자에게 알리고, 문제가 있다면 디버깅을 위한 로그를 제공
 - **배포** : 모든 테스트가 성공하면 Jenkins는 소프트웨어를 자동으로 배포함

Jenkins 설치 전 설정

- Jenkins 는 최소 2GB 의 메모리를 요구
- 이를 충족시키지 못하는 경우, 리눅스 swap 메모리를 통해 RAM 확장
 - swap memory
 - 컴퓨터의 가상 메모리 관리에서 사용되는 공간으로, 물리적 RAM(Random Access Memory)이 부족할 때 디스크의 일부를 메모리처럼 사용하는 영역
 - RAM이 완전히 부족한 경우 시스템의 크래시를 방지할 수 있음
 - 명령어
 - 리눅스 OS의 메모리 조회: `free -h`
 - swap 파일 생성하기: `sudo dd if=/dev/zero of=/swapfile bs=1M count=2048`
 - swap 메모리를 swap 파일로 포맷: `sudo mkswap /swapfile`
 - swap 파일의 읽기 쓰기 권한 업데이트: `sudo chmod 600 /swapfile`
 - swap 공간에 swapfile 추가: `sudo swapon /swapfile`
 - 재부팅 시에도 swap 활성화하는 방법

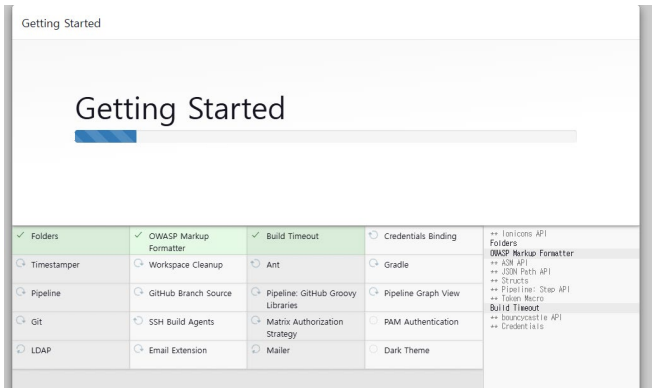
```
sudo nano /etc/fstab
```

```
/swapfile none swap defaults 0 0
```

Jenkins 설치

- Docker 설치 권장 (직접 설치 시 부가적인 설치 및 환경설정 작업이 많음)
- `docker pull jenkins/jenkins`
- `docker run -d -v jenkins_home:/var/jenkins_home -p 12345:8080 -p 50000:50000 --restart=on-failure --name jenkins-server jenkins/jenkins`
- 포트 설정
 - 외부 접속 가능한 인터페이스 포트 : 12345 (변경 가능)
 - 분산 빌드를 위한 포트 : 50000 (생략 가능)

- Plugin 설정
 - Suggested plugin install



```
*****  
*****  
*****  
Jenkins initial setup is required. An admin user has been created a  
Please use the following password to proceed to installation:  
  
bdf7aed94bab46deb86135d4aaf5a51f  
  
This may also be found at: /var/jenkins_home/secrets/initialAdminPa  
*****  
*****  
*****
```



Jenkins 설정

- admin 계정 생성하기

Create First Admin User

계정명

암호

암호 확인

이름

이메일 주소

Jenkins 설정

- Instance Configuration
 - 실제로 접속에 활용될 주소로 설정
 - nginx proxy manager 에 등록 후 hostname 사용 가능

Instance Configuration

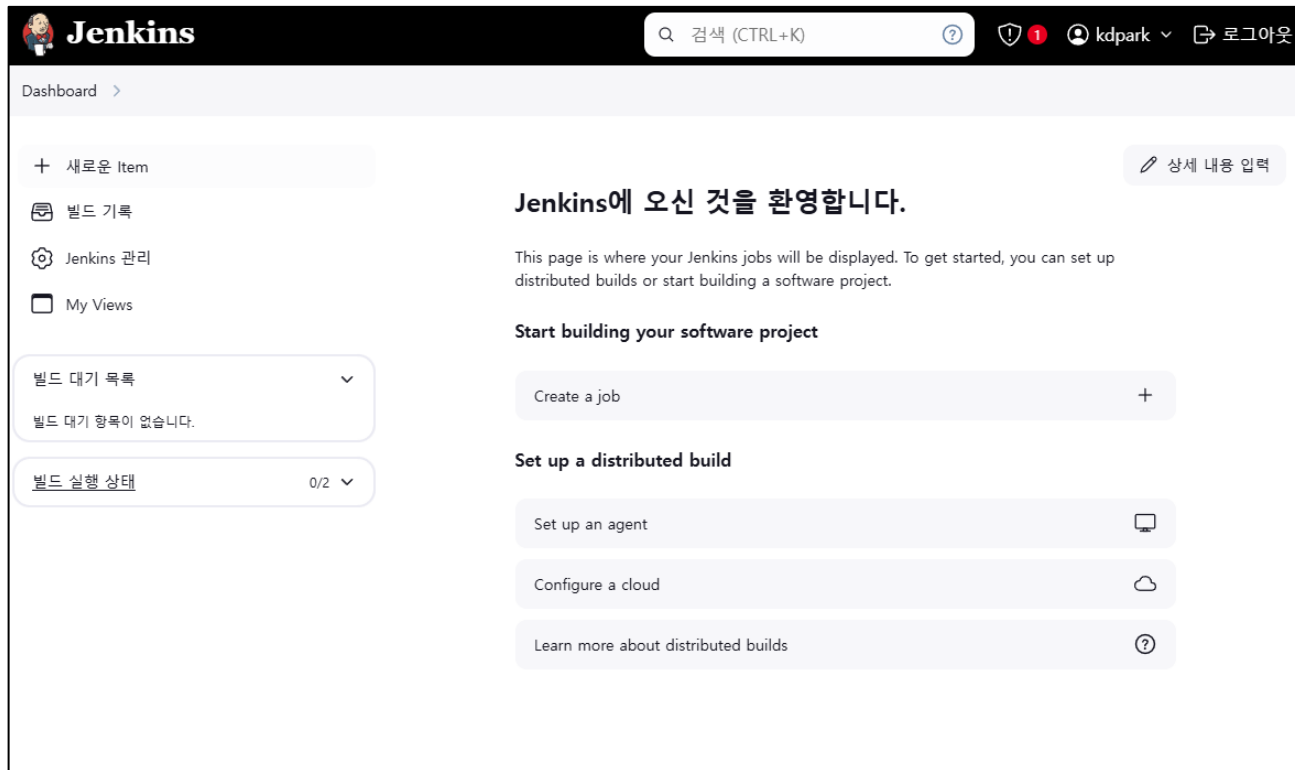
Jenkins URL:

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

Jenkins 접속

- 접속 화면



Jenkins 플러그인 설치

- 플러그인 설치
 - Jenkins 관리 > System Configuration > Plugins
 - Available plugins
 - NodeJS Plugin 검색
 - 체크 후 Install
 - 추가 설치
 - ssh agent : Jenkins 가 ssh 로 서버에 접속하고 배포 명령어를 수행할 수 있도록 함
 - discord notifier : discord 채널을 통해 빌드 상황을 전달 받을 수 있음

Jenkins 플러그인 설치

- 플러그인 설치
 - Jenkins 관리 > Tools

NodeJS

Name

my-nodejs

☒ Install automatically ?

Install from nodejs.org

Version

NodeJS 23.2.0

For the underlying architecture, if available, force the installation of the 32bit package. Otherwise the build will fail

☐ Force 32bit architecture

Global npm packages to install

Specify list of packages to install globally -- see npm install -g. Note that you can fix the packages version by using the syntax `packageName@version`

pm2

Global npm packages refresh hours

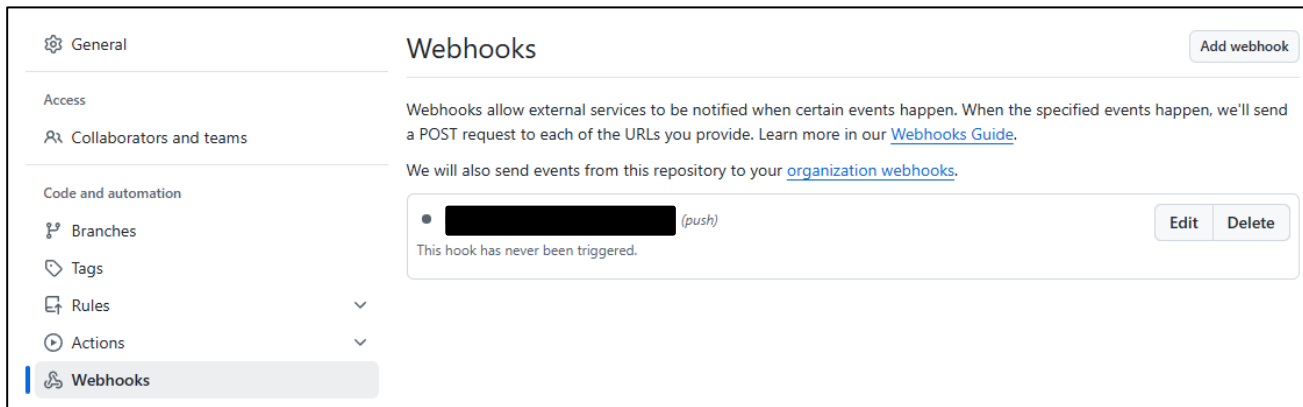
Duration, in hours, before 2 npm cache update. Note that 0 will always update npm cache

72

Add Installer

Jenkins 빌드 아이템 생성

- github 설정
 - 특정 **branch** 에 **push** 되면 자동으로 코드를 Jenkins 로 보내서 빌드 및 배포를 진행함
 - Webhook 등록
 - Add webhook



Jenkins 빌드 아이템 생성

- github 설정
 - 특정 **branch** 에 **push** 되면 자동으로 코드를 Jenkins 로 보내서 빌드 및 배포를 진행함
- Webhook 등록
 - payload URL
 - jenkins 주소 뒤에 `/github-webhook/` 추가
 - Content type
 - `_application/json_`
 - SSL verification
 - Jenkins 를 HTTPS 로 배포할 때 활성화

We will also send events from this repository

✓ [http://k\[redacted\]/gi...](http://k[redacted]/gi...) (push)
Last delivery was successful.

Payload URL *

Content type *

Secret

SSL verification
By default, we verify SSL certificates when delivering payloads.
☐ Enable SSL verification ☒ Disable (not recommended)

Which events would you like to trigger this webhook?
☒ Just the push event.
☐ Send me everything.
☐ Let me select individual events.

☒ **Active**
We will deliver event details when this hook is triggered.

Jenkins 빌드 아이템 생성

- github 설정
 - 특정 branch 에 push 되면 자동으로 코드를 Jenkins 로 보내서
 - Jenkins 에서 git clone 을 수행하기 위해
 - private repo 의 경우, Personal Access Token 을 발급하고 Jenkins 에 등록해야함
 - Personal Access Token 을 발급 절차 : <https://1minute-before6pm.tistory.com/52>
 - Jenkins 에 등록
 - Jenkins 관리 클릭 > Credentials > (Global) > Add Credentials
 - username : GITHUB ID
 - password : Personal Access Token 입력
 - ID : 추후 해당 인증을 불러오기 위한 고유 이름

New credentials

Kind

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

kdpark-phd

☐ Treat username as secret ?

Password ?

.....

ID ?

github_kdpark_id

Description ?

Create

Jenkins 빌드 아이템 생성

- github 설정
 - 특정 branch 에 push 되면 자동으로 코드를 Jenkins 로 보내서 빌드 및 배포를 진행함
- 새로운 아이템 추가
 - pipeline 선택
 - GitHub project 체크
 - URL 등록
 - Github hook trigger for GITScm polling 체크

The screenshot shows the Jenkins configuration page for a new item. The 'GitHub project' checkbox is checked. Below it, the 'Project url' field is visible with a redacted URL. There is a '고급' (Advanced) button with a dropdown arrow. Below the 'Project url' field, there are several unchecked checkboxes: 'Pipeline speed/durability override', 'Preserve stashes from completed builds', 'Throttle builds', '오래된 빌드 삭제' (Delete old builds), and '이 빌드는 매개변수가 있습니다' (This build has parameters). Under the 'Build Triggers' section, there are three checkboxes: 'Build after other projects are built', 'Build periodically', and 'GitHub hook trigger for GITScm polling', which is checked.

☒ GitHub project

Project url ?

[Redacted URL]

고급 ▾

☐ Pipeline speed/durability override ?

☐ Preserve stashes from completed builds ?

☐ Throttle builds ?

☐ 오래된 빌드 삭제 ?

☐ 이 빌드는 매개변수가 있습니다 ?

Build Triggers

☐ Build after other projects are built ?

☐ Build periodically ?

☒ GitHub hook trigger for GITScm polling ?

Jenkins 빌드 아이템 생성

- github 설정
 - 특정 branch 에 push 되면 자동으로 코드를 Jenkins 로 보내서 빌드 및 배포를 진행함
 - pipeline script 작성
 - git branch : 빌드할 branch
 - credentialsId : 인증 시 입력한 ID
 - url : HTTPS 방식의 git 주소

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git branch: 'main',
           credentialsId: 'github_kdpark_id',
           url: 'https://github.com/kw-ic-web/express_skeleton_html.git'
      }
    }
  }
}
```

Jenkins 빌드 아이템 생성

- github 설정

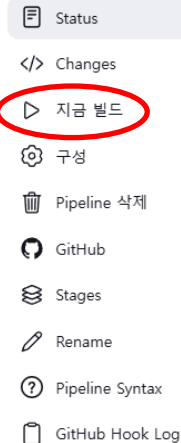
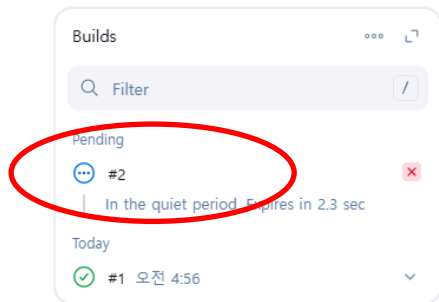
- 특정 branch 에 push 되면 자동으로 코드를 Jenkins 로 보내서 빌드 및 배포를 진행함

- 테스트 1

- "지금 빌드" 클릭 : github clone 을 시도

- 테스트 2

- main branch 의 코드 수정 후 push



✓ #1 (2024. 11. 19. 오전 4:56:35)

🕒 사용자 **kdpark** 에 의해 시작됨

🕒 This run spent:

- 0.14 sec waiting;
- 16 sec build duration;
- 16 sec total from scheduled to completion.



Revision: bde90a91fe09a66e702fcc7cc03646b53af075e8

Repository: https://github.com/kw-ic-web/express_skeleton_html.git

- refs/remotes/origin/main



변경사항 없음.

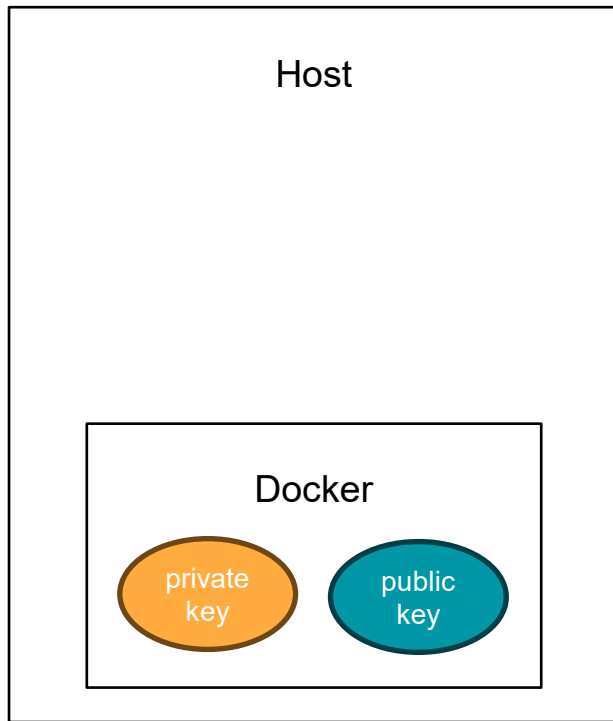
Jenkins 빌드 아이템 생성

- github 설정
 - 특정 branch 에 push 되면 자동으로 코드를 Jenkins 로 보내서 빌드 및 배포를 진행함
 - clone 위치의 확인
 - /var/lib/docker/volumes/jenkins_home/_data/workspace/
 - 관리자 권한이 필요 (해당 경로를 바로 배포하면 정적 파일에 대한 접근이 어려움)
 - 여러 가지 대안 존재
 - docker container 에서 host 의 user home 폴더로 copy 한 후
 - ① 직접 배포 (pm2 또는 nohup)
 - ② docker 로 만들어서 배포
 - 위를 자동화하기 위해 ssh agent 필요

Jenkins 빌드 아이템 생성

- 사전 준비
 - sudo 없이 docker 명령어 수행 가능하도록 변경
 - `sudo usermod -aG docker [username]`
 - 재접속 필요
 - ssh key 발급 및 설정
 - Jenkins 컨테이너 내부에서 ssh 키 를 생성
 - `$ docker exec -it [jenkins] bash`
 - `$ ssh-keygen -t rsa -b 4096`

```
kw.idea.14@ubuntu:~$ docker exec -it jenkins-server bash
jenkins@00c252fb62da:/$ ls
bin boot dev etc home lib media mnt opt proc root run sbin srv sys tmp usr var
jenkins@00c252fb62da:/$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/var/jenkins_home/.ssh/id_rsa):
Created directory '/var/jenkins_home/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /var/jenkins_home/.ssh/id_rsa
Your public key has been saved in /var/jenkins_home/.ssh/id_rsa.pub
```



Jenkins 빌드 아이템 생성

- 사전 준비

- 공개키 내용 확인

- \$ cat /var/jenkins_home/.ssh/id_rsa.pub

- 해당 공개키 복사 후 host 의 authorized_keys 에 추가

- 복사 후 exit

- \$ echo “복사값” >> ~/.ssh/authorized_keys

```
kw.idea.14@ubuntu:~$ echo "ssh-rsa AAAA
JA6k2f03eBfTQ1vL3ZRoiCzrTyZ0EG4Q4npmsF
PNh1nU8B2ILZJL9z6Iwd9EBIXaVqpDaCqgIJOL
Pp5Ln+awcLPsg+lGRN06hgbODI3mWMZq/1lMsd
yx+UMYFRnCLwLvdjyWfouexuUbZxB1pJf2Xgkz
GxEyI4Gv3JpbGtxbiGKt4kRy6Xhtdj0VWGoFbA
RlnFdzvC7RjkQ/MxupOXDK8NA+jHKU/nWaiiKQ
b0s4kjlCQo8smwDwWL0lmQzfHl60lH9gBhrSP1
62da" >> ~/.ssh/authorized_keys
```

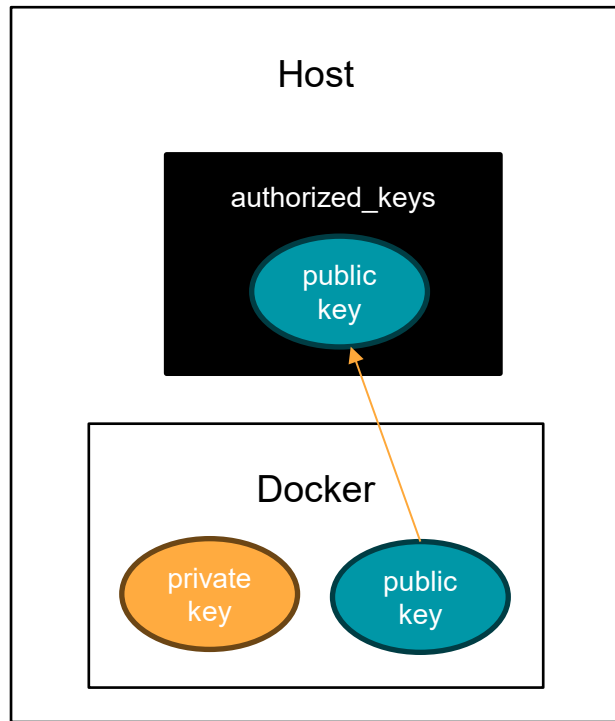
- 권한 조정

- \$ chmod 700 ~/.ssh

- \$ chmod 600 ~/.ssh/authorized_keys

- 다시 도커에서 ssh 접속이 잘 되는지 확인

- \$ ssh -o StrictHostKeyChecking=no [ID]@[HOST] -p [PORT]



Jenkins 빌드 아이템 생성

- 사전 준비
 - 도커 컨테이너 내부에 rsync 설치
 - 보다 효율적인 파일 전송 및 동기화 제공
 - `$ docker exec -it -u root jenkins-server bash`
 - `$ apt-get update`
 - `$ apt-get install rsync`

Jenkins 빌드 아이템 생성

- Jenkins 에 SSH 인증 등록
 - Jenkins 관리 > Credentials > System 탭의 (Global)
 - Add Credentials
 - Kind
 - SSH Username with private key
 - ID
 - pipeline 에서 활용될 ID
 - Username
 - Host 의 사용자이름
 - Private key
 - Jenkins docker container 내에서
 - `$ cat /var/jenkins_home/.ssh/id_rsa`
 - 내용 복사

The screenshot shows the 'Add Credentials' form in Jenkins. The 'Kind' is set to 'SSH Username with private key'. The 'Scope' is 'Global (Jenkins, nodes, items, all child items, etc)'. The 'ID' is 'docker_ssh'. The 'Description' is empty. The 'Username' is 'kw.idea.14'. The 'Treat username as secret' checkbox is unchecked. The 'Private Key' section has 'Enter directly' selected. The 'Key' field contains a long base64-encoded string. An orange arrow points from the '내용 복사' (Copy content) step in the list to the 'Key' field.

Kind

SSH Username with private key

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

ID ?

docker_ssh

Description ?

Username

kw.idea.14

☐ Treat username as secret ?

Private Key

☒ Enter directly

Key

Enter New Secret Below

```
■HJ / ZPukb3ANb4xrh4Xnj D1 bnbA9Ub fASTwUp/ YkY/ Pf-w / bBcHr3nr bblan0P / XsJ / NVJ Kf-  
Sxj 948CdwsvNYZl KcDyW3t SXucZBWl 07mzD6Da09S23nm09qmS9zLUGT vEckX327FDg5zq  
4XSFaygn2pedZ/ yKHai fgtCU+ofAj XpDwUS+ l cHsGh0mr CMgz/ oadHT cWndd9NHai sBwXET  
pLGj BdHy l 9MAAAAUamYua2l ucDAwMGMyNTJmYj YyZGEBAGMEBQYH
```

Jenkins 빌드 아이템 생성

- github 설정
 - 특정 branch 에 push 되면 자동으로 코드를 Jenkins 로 보내서 빌드 및 배포를 진행함
 - 파이프라인 (ssh agent 구성)

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git branch: 'main',
            credentialsId: 'github_kdpark_id',
            url: 'https://github.com/kw-ic-web/express_skeleton_html.git'
      }
    }
    stage('Copy and deploy') {
      steps {
        sshagent(credentials: ['docker_ssh']) {
          sh """
            rsync -az -e "ssh -p [port]" ./ [username]@[host]:[destination_path]
            ssh -o StrictHostKeyChecking=no -p [port] [username]@[host] '
              cd ./test_docker
              ls
            '
          """
        }
      }
    }
  }
}
```

Jenkins 빌드 아이템 생성

- 1) pm2 를 활용한 직접 배포
 - 파이프라인

```
sshagent(credentials: ['docker_ssh']) {  
    sh """  
        rsync -az -e "ssh -p [port]" ./ [username]@[host]:[destination_path]  
        ssh -o StrictHostKeyChecking=no -p [port] [username]@[host] '  
            cd ./test docker  
            npm i  
            if pm2 describe my-express > /dev/null; then  
                pm2 restart my-express  
            else  
                PORT=21481 pm2 start ./bin/www --name my-express  
            fi  
        '  
    """  
}
```

- 테스트
 - github 의 readme 파일을 변경하여 등록한 branch 에 push
 - 자동으로 배포가 되면 성공

Jenkins 빌드 아이템 생성

- 2) Docker 빌드 및 배포
 - docker 설정 파일 생성

[express_skeleton_html](#) / Dockerfile 



kdpark-phd Create Dockerfile

Code

Blame

8 lines (8 loc) · 119 Bytes

```
1  # Dockerfile
2  FROM node:20
3  WORKDIR /app
4  COPY package*.json ./
5  RUN npm install
6  COPY . .
7  EXPOSE 3000
8  CMD ["npm", "start"]
```


Jenkins 빌드 아이템 생성

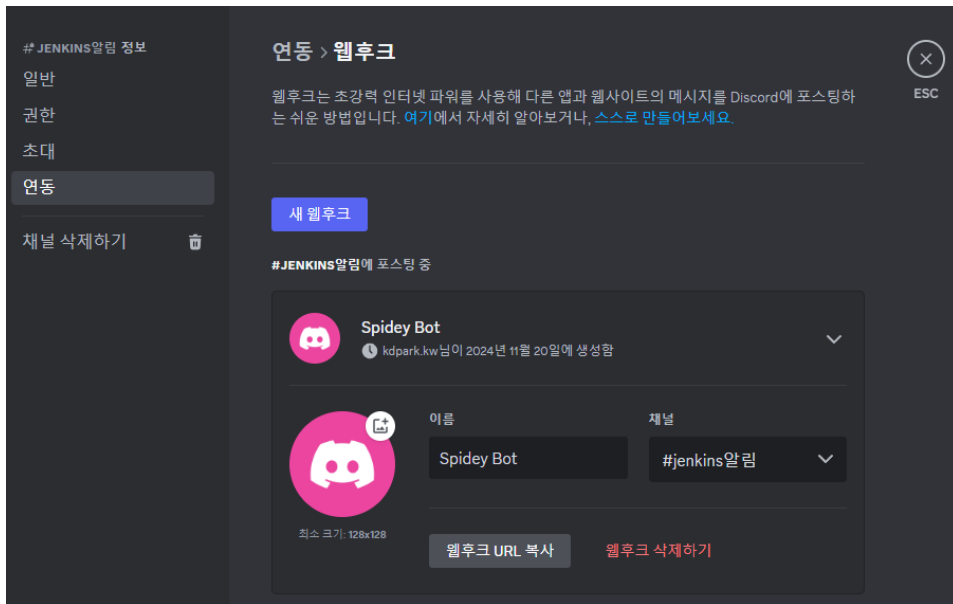
- 파이프라인에 Docker 배포 명령어 추가

```
sshagent(credentials: ['docker_ssh']) {  
    sh """  
        rsync -az -e "ssh -p [port]" ./ [username]@[host]:[destination_path]  
        ssh -o StrictHostKeyChecking=no -p [port] [username]@[host] '  
            cd ./test docker  
            docker build -t my-express .  
            docker stop my-express  
            docker rm my-express  
            docker run -d --name my-express -p [accessport]:[innerPort] my-express  
        '  
    """  
}
```

- 테스트
 - github 의 readme 파일을 변경하여 등록한 branch 에 push
 - 자동으로 배포가 되면 성공

Jenkins 배포 완료 알림 받기 (feat. Discord)

- Discord 를 사용하여, 배포 완료 시 알림을 받는 기능
 - discord notifier plugin 설치
- 알림을 받고자 하는 채널에서 웹후크 설정



Jenkins 배포 완료 알림 받기 (feat. Discord)

- Webhook 을 Jenkins Credentials 에 추가

– discord webhook

– pipeline 에서 사용할 ID

Dashboard > Jenkins 관리 > Credentials > System > Global credentials (unrestricted) >

New credentials

Kind

Secret text

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Secret

.....

ID ?

discord_webhook

Description ?

|

Create

Jenkins 배포 완료 알림 받기 (feat. Discord)

- 파이프라인 진행 결과에 따른 수행
 - post 내에 적절한 조건을 선택

```
pipeline {
  ...
  stages {
    ...
  }
  post {
    always {
      echo '항상 실행'
    }
    success {
      echo '빌드가 성공한 경우 실행'
    }
    failure {
      echo '빌드 실패한 경우 실행'
    }
    unstable {
      echo '빌드 상태 불안한 경우 실행'
    }
    changed {
      echo '현재 빌드상태가 이전 빌드상태와 달라진 경우 실행'
    }
    unsuccessful {
      echo '성공' 상태가 아닌 경우 실행'
    }
  }
}
```

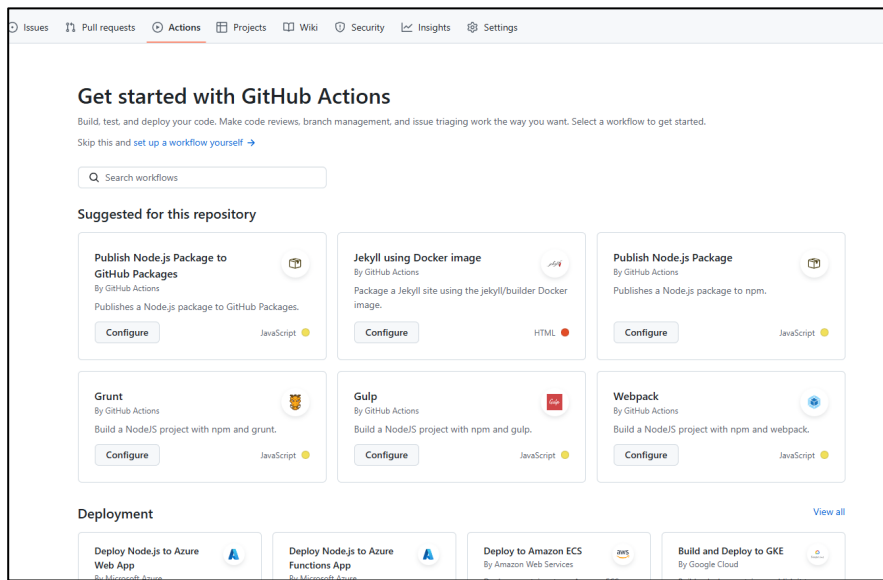
Jenkins 배포 완료 알림 받기 (feat. Discord)

- 예시

```
post {
    success {
        withCredentials([string(credentialsId: 'discord_webhook', variable: 'DISCORD')]) {
            discordSend description: ""
            제목 : ${currentBuild.displayName}
            결과 : ${currentBuild.result}
            실행 시간 : ${currentBuild.duration / 1000}s
            "",
            link: env.BUILD_URL, result: currentBuild.currentResult,
            title: "${env.JOB_NAME} : ${currentBuild.displayName} 성공",
            webhookURL: "$DISCORD"
        }
    }
    failure {
        withCredentials([string(credentialsId: 'discord_webhook', variable: 'DISCORD')]) {
            discordSend description: ""
            제목 : ${currentBuild.displayName}
            결과 : ${currentBuild.result}
            실행 시간 : ${currentBuild.duration / 1000}s
            "",
            link: env.BUILD_URL, result: currentBuild.currentResult,
            title: "${env.JOB_NAME} : ${currentBuild.displayName} 실패",
            webhookURL: "$DISCORD"
        }
    }
}
```

Advanced Topic

- Jenkin Pipeline 의 syntax
 - <https://www.jenkins.io/doc/book/pipeline/syntax/>
- Jenkins 대신 github action 을 활용 가능



Thank you

Q & A